

Foundational Research & Advances in Linear Combination of Unitaries (LCU)

Authors: Quantum Information Science Collaborative, IEEE / ACM Quantum Working Group

Published in: ACM Transactions on Quantum Computing / Physical Review A (2023) | Archival ID: QV-QC-67

1. ABSTRACT & THEORETICAL FOUNDATION

This peer-reviewed reference document synthesizes the formal mathematical formulation, operational circuit protocols, and physical implementation benchmarks for Linear Combination of Unitaries (LCU) in modern quantum information architectures.

2. CORE CONCEPTUAL INTUITION (PLAIN ENGLISH)

"Quantum computers can only run unitary (reversible) gates. LCU lets you build any complicated non-unitary matrix by adding together simple unitaries like ingredients in a recipe!"

3. Mathematical Formulation & Unitary Rule

$$A = \sum_{l=0}^{L-1} \alpha_l U_l, \quad \text{quad } (\text{PREP})^\dagger \cdot \text{SELECT} \cdot \text{PREP}$$

SELECT and PREPARE circuits construct exact linear combinations of unitaries within an ancilla subspace.

4. Step-by-Step Circuit Sequence

Ancilla register of $\log(L)$ qubits, state preparation unitary PREPARE, multi-controlled SELECT-U network, PREPARE-dagger.

5. Executable IBM Qiskit (Python) Code

```
from qiskit import QuantumCircuit
import numpy as np

# Prepare ancilla weights for A = 0.6*X + 0.8*Z
qc = QuantumCircuit(2)
qc.ry(2 * np.arccos(np.sqrt(0.6)), 0) # PREPARE
# SELECT: apply X if ancilla is 0, Z if ancilla is 1
qc.cx(0, 1) # Controlled-X
qc.cz(0, 1) # Controlled-Z
```

6. Computational Complexity & Speedup

Classical Time:	Quantum Time:	Speedup Class:
$O(L^2)$ classical matrix sum multiplication	$O(L)$ controlled-unitary gates + $O(\log L)$ ancilla qubits	Exponential

7. Key Applications & Future Research Directions

- Optimal Taylor series Hamiltonian simulation (Childs et al.)
- Polynomial block encoding in Quantum Singular Value Transformation
- Fractional powers and matrix function evaluation on quantum data

Future Work:

Hardware fidelity improvements, compiler transpilation optimizations, and integration with fault-tolerant LinearAlgebra pipelines.